Cryptography Lab on Ubuntu

Beginner practical: encryption, decryption, hashing and digital signatures using OpenSSL

Objective: Practise core cryptography concepts on Ubuntu and observe how encryption, hashing and signatures behave.

Requirements: Ubuntu 22.04/24.04 (or a recent Ubuntu VM), terminal access and OpenSSL.

1. Prepare the lab environment

Open Terminal, check OpenSSL, install it if required, and create a dedicated practice directory.

```
openssl version
sudo apt update
sudo apt install openssl -y

mkdir -p ~/crypto_lab
cd ~/crypto_lab

echo "Hello Students! This is my first cryptography lab." > message.txt
cat message.txt
```

Expected: The OpenSSL version and sample message are displayed.

2. Symmetric encryption and decryption (AES-256-CBC)

Concept: Symmetric encryption uses the same secret password/key to encrypt and decrypt. OpenSSL derives the encryption key from the password entered.

Encrypt the file

```
openssl enc -aes-256-cbc -salt -pbkdf2 \
  -in message.txt -out message.enc
```

Enter a password when prompted. The encrypted data is saved in **message.enc** and will not look like readable text.

Decrypt the file

```
openssl enc -aes-256-cbc -d -pbkdf2 \
  -in message.enc -out decrypted.txt

cat decrypted.txt
```

Expected: The original message appears.

Try an incorrect password

```
openssl enc -aes-256-cbc -d -pbkdf2 \
  -in message.enc -out wrong.txt
```

Enter a different password. Decryption should fail or produce invalid output. CBC encryption alone does not authenticate ciphertext, so a wrong password is not guaranteed to produce a clean error.

3. Asymmetric encryption and decryption (RSA)

Concept: RSA uses a public/private key pair. The public key encrypts in this exercise; the private key decrypts. These commands explicitly use OAEP padding with SHA-256.

Generate the key pair

```
openssl genpkey -algorithm RSA -out private.pem \
  -pkeyopt rsa_keygen_bits:2048

openssl pkey -in private.pem -pubout -out public.pem
```

Encrypt and decrypt a short message

```
printf "Secret message for RSA lab" > rsa_message.txt

openssl pkeyutl -encrypt -pubin -inkey public.pem \
  -in rsa_message.txt -out rsa_encrypted.bin \
  -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256

openssl pkeyutl -decrypt -inkey private.pem \
  -in rsa_encrypted.bin -out rsa_decrypted.txt \
  -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256

cat rsa_decrypted.txt
```

Expected: The original short message is displayed. RSA is for relatively small data; practical systems commonly use hybrid encryption, where symmetric encryption protects the data and public-key cryptography protects the symmetric key.

4. Hashing with SHA-256

Concept: A cryptographic hash produces a fixed-length digest. Hashing is one-way and is not designed to be decrypted.

```
openssl dgst -sha256 message.txt

echo "A small change in the message" >> message.txt
openssl dgst -sha256 message.txt
```

Compare the two digests. A small change in the file should produce a different hash.

5. Digital signatures with RSA

Concept: A digital signature helps verify message integrity and that the signature was created using the corresponding private key. The public key verifies it.

Create a signature

```
printf "This message is digitally signed." > signed_message.txt

openssl dgst -sha256 -sign private.pem \
  -out signature.bin signed_message.txt
```

Verify the signature

```
openssl dgst -sha256 -verify public.pem \
  -signature signature.bin signed_message.txt
```

Expected: `Verified OK`

Modify the message and verify again

```
echo "Modified message" >> signed_message.txt

openssl dgst -sha256 -verify public.pem \
  -signature signature.bin signed_message.txt
```

Expected: Verification failure, because the signed content has changed.

6. Compare the concepts

Concept	Purpose	Key idea
Symmetric encryption	Confidentiality	Same secret key for encryption and decryption
Asymmetric encryption	Confidentiality / key exchange	Public and private key pair
Hashing	Integrity checking	One-way fixed-length digest; no decryption
Digital signature	Integrity and authenticity	Private key signs; public key verifies

7. Student practical checklist

- ■ Create a text file containing your name and roll number.
- ■ Encrypt the file using AES-256-CBC and decrypt it.
- ■ Try decrypting with a different password and note the result.
- ■ Generate an RSA private key and public key.
- ■ Encrypt and decrypt a short message using RSA-OAEP with SHA-256.
- ■ Generate a SHA-256 hash of a file.
- ■ Modify the file and compare the new hash.
- ■ Create an RSA digital signature for a message.
- ■ Verify the signature using the public key.
- ■ Modify the signed message and verify it again.

8. Safety notes and discussion

• This is a learning exercise, not a production security configuration.

• AES-CBC via the OpenSSL enc command does not authenticate ciphertext. For real applications, use an authenticated-encryption design such as AES-GCM through an appropriate supported API or tool.

• Keep private keys and passwords secret. Do not use confidential data in a classroom lab.

• Use a disposable practice directory. The private key created here is unencrypted; protect it and delete it when the lab is complete.

Discussion: Why is hashing not encryption? Why can a public key be shared while a private key must be protected? What does signature verification failure indicate?

9. Learning outcomes and duration

After completing this lab, students should be able to distinguish symmetric and asymmetric encryption, encrypt and decrypt sample data, generate and compare SHA-256 hashes, and create and verify a digital signature. Suggested duration: approximately 90 minutes.

Embedded-systems connection: These concepts are foundations for secure boot, firmware integrity checks, protected key storage, encrypted storage and TLS-based communication in embedded Linux.